

Reinforcement Learning for Sepsis Treatment Optimization

Policy Learning in Discrete and Continuous Clinical Environments

Authors: Andreea Roica, Marisa Esteves, Miguel Caramelo, Oliver Kain

Student IDs: 20250361, 20250348, 20250378, 20250401

Professor: Inês Castelhana

Course: Reinforcement Learning

University: NOVA Information Management School

Date: June 05, 2026

Contents

1	Abstract (optional)	3
2	Introduction	3
3	Environment and Problem Setup	3
3.1	Configuration A: Discrete State Space	4
3.2	Configuration B: Continuous Observation Space	4
4	Methodology	5
4.1	Evaluation Metrics	5
4.2	Configuration A Algorithms	5
4.2.1	Policy Iteration & Value Iteration	5
4.2.2	Training Setup	5
4.3	Configuration B Algorithms	5
4.3.1	Double DQN	5
4.3.2	Proximal Policy Optimization (PPO)	6
4.3.3	Training Setup	6
5	Results and Evaluation	7
5.1	Configuration A Results	7
5.1.1	Evaluation Results	7
5.1.2	Convergence Analysis	7
5.1.3	Computational Performance	8
5.2	Configuration B Results	8
5.2.1	Training Curves	8
5.2.2	Performance Comparison	8
5.2.3	Deep RL Behavior Analysis	8
5.3	Configuration A vs Configuration B Comparison	8
6	Creative Extension: Policy Agreement & Ensemble Analysis	8
6.1	Motivation	8
6.2	Method	9
6.3	Results and Discussion	9
7	Discussion	10

8	Conclusion	10
A	Figures	11
B	Tables	15

1 Abstract (optional)

Short summary: problem motivation (sepsis treatment), RL approach, config A vs config B, algorithms used, main findings, creative extension contribution.

2 Introduction

Sepsis is one of the leading causes of mortality in intensive care units (ICUs) worldwide and represents a major challenge in modern healthcare. The condition arises from the body’s dysregulated response to infection, often leading to organ failure and severe physiological instability. Treating sepsis requires clinicians to continuously make complex decisions regarding intravenous (IV) fluids and vasopressor administration in order to stabilize the patient while avoiding harmful side effects caused by excessive treatment. Because patient responses vary significantly and outcomes are often delayed, determining the optimal treatment strategy is highly challenging.

This setting naturally motivates the application of Reinforcement Learning (RL). In the context of sepsis treatment, the RL agent learns treatment policies directly from historical ICU trajectories, aiming to maximize patient survival while minimizing unnecessary interventions.

In this project, we investigate the application of RL methods to the ICU-Sepsis-v2 benchmark environment, which is derived from the real-world MIMIC-III clinical database. The project is divided into two distinct configurations that reflect different levels of environmental complexity. Configuration A uses a discrete state representation, allowing the application of classical tabular reinforcement learning methods and model-based approaches. Configuration B introduces a continuous high-dimensional observation space composed of physiological measurements, requiring the use of deep reinforcement learning and function approximation techniques.

The primary objective of this work is to compare the behavior and performance of reinforcement learning algorithms across both environments.

To extend the analysis, we developed a creative extension focused on policy agreement and ensemble behavior. By analyzing agreement and disagreement patterns between learned policies, we aim to better understand uncertainty and robustness in clinical decision-making scenarios.

3 Environment and Problem Setup

The experiments were conducted using the ICU-Sepsis-v2 environment, a benchmark designed for offline reinforcement learning in healthcare applications. The environment is based on patient trajectories extracted from the MIMIC-III clinical database and simulates the treatment of sepsis patients in intensive care units. Each episode corresponds to a single patient trajectory, where the agent sequentially observes the patient’s condition and selects a treatment action at every timestep.

The action space is discrete and composed of 25 possible actions, representing combinations of vasopressor dosage and IV fluid administration levels. The objective of the agent is to learn a treatment policy that maximizes patient survival outcomes. At the end of each episode, the agent receives a terminal reward of +1 if the patient survives and 0 otherwise.

3.1 Configuration A: Discrete State Space

In Configuration A, the environment provides a discrete representation of the patient state. Each observation corresponds to an integer between 0 and 715, resulting in a finite state space of 716 possible clinical states. Since both the state and action spaces are discrete and relatively small, classical tabular reinforcement learning methods become computationally feasible. In particular, the complete transition and reward matrices of the underlying Markov Decision Process (MDP) are accessible, enabling the use of model-based algorithms.

To better understand the environment before training the agents, an exploratory analysis of the MDP structure was conducted. The transition dynamics were relatively sparse, with only around 12% of all possible state-action-next-state transitions having non-zero probability. However, the environment exhibited substantial stochasticity and uncertainty. On average, each state-action pair could transition to approximately 85 possible next states, indicating a high branching factor and variability in patient outcomes following identical clinical interventions.

The reward structure of the environment is sparse and primarily terminal-based. Although most states are connected to at least some positive reward signal (687 states), the average expected reward per state remains low (0.1133), suggesting that survival signal is widely reachable across the state space but remains weak and unevenly distributed.

Additional analysis of the reward matrix revealed that approximately 72% of states contain a clearly dominant action with the highest expected reward. However, the distribution of optimal actions across states was highly imbalanced, with some actions appearing optimal far more frequently than others (Figure 1).

A random-policy baseline was also evaluated in order to establish a minimum performance threshold for the learning agents. It provided a reference point for determining whether the learned agents extract meaningful treatment strategies beyond random exploration.

3.2 Configuration B: Continuous Observation Space

In Configuration B, the environment provides a continuous representation of the patient state, composed of 47 normalized physiological measurements. The same exact state is unlikely to be observed repeatedly, which motivates the use of function approximation.

The action space remains discrete, with 25 possible actions. Although, because the observation space is continuous, the full tabular transition matrix used in Configuration A is no longer directly available or practical to analyze. As a result, the initial analysis focused on the structure of the observation vector, the clinical dimensionality of the input, the action space, and the additional environment flags used to monitor specific sources of uncertainty.

The Configuration B environment also includes wrappers that introduce additional clinical realism: `AcuteEventEnv`, `EpisodicMissingObsEnv`, and `EpisodicNoisyObsEnv`. These wrappers generate episodes with environment-defined acute clinical events, missing features, and noisy observations.

As in Configuration A, a random-policy baseline (50,000 episodes) was evaluated to establish a minimum performance threshold.

4 Methodology

4.1 Evaluation Metrics

Several metrics were used to assess the quality and efficiency of the learned policies.

The primary performance measure was the mean return, representing the average cumulative reward obtained during evaluation. Since the environment includes a treatment-intensity penalty, this metric reflects both patient survival and treatment efficiency.

The survival rate was additionally reported as the percentage of episodes ending in patient survival, providing a clinically interpretable measure of policy quality. Furthermore, the mean episode length was recorded to analyse how long patient trajectories remained active before reaching a terminal outcome.

To evaluate the optimisation process itself, convergence speed was measured using the number of iterations required to reach a stable solution, while policy stability was assessed by tracking the number of states whose selected action changed between successive iterations.

UPDATE WITH INFO FROM CONFIG B IF SOMETHING IS WRONG OR ADDED

4.2 Configuration A Algorithms

4.2.1 Policy Iteration & Value Iteration

Considering the nature of Configuration A, described in Section 3.1, Policy Iteration and Value Iteration were selected as the primary algorithms for this configuration. Both methods are guaranteed to converge to the optimal policy in finite MDPs and provide a strong benchmark for the best achievable performance in the tabular setting. Additionally, comparing the policies and convergence behaviour of both algorithms allows the correctness of the implementation to be validated while highlighting potential differences in computational efficiency.

4.2.2 Training Setup

Experiments were conducted using a treatment-intensity penalty of $\lambda = 0.02$ and a SOFA bias of 5.0. Both Policy Iteration and Value Iteration were executed with a discount factor of $\gamma = 1.0$. Convergence was determined using the maximum change in the value function, with a threshold of $\theta = 10^{-8}$ for Policy Iteration and $\theta = 10^{-4}$ for Value Iteration. Runtime and convergence statistics were recorded to compare the computational efficiency of the algorithms.

4.3 Configuration B Algorithms

4.3.1 Double DQN

Double DQN was selected as the value-based deep RL method for Configuration B. Since the agent observes a 47-dimensional continuous clinical vector, a tabular Q-table is not feasible. Instead, Double DQN approximates $Q(s,a)$ with a neural network, allowing the agent to gener-

alise across similar patient states while still handling the discrete action space of 25 treatment combinations.

The method uses experience replay and a target network to improve training stability. Experience replay stores transitions and samples mini batches uniformly at random, reducing temporal correlation between consecutive observations. The buffer size controls how much old experience is retained. The target network provides more stable bootstrap targets. Double DQN further reduces Q-value overestimation by using the online network to select the next action and the target network to evaluate it. This makes it suitable for a stochastic clinical environment with continuous observations and discrete treatment decisions.

For Double DQN, the search space included the learning rate, exploration fraction, minimum epsilon, batch size, replay buffer size, target-network update frequency, number of gradient steps and hidden-layer size.

4.3.2 Proximal Policy Optimization (PPO)

PPO was selected as the policy-gradient method for Configuration B. Unlike Double DQN, which learns action-value functions, PPO directly optimizes a parameterized policy using an actor-critic architecture. The actor decides which action to take, while the critic estimates the value of the current state to evaluate those actions.

As an on-policy method, PPO learns exclusively from fresh rollouts collected by the current policy, eliminating the need for an experience replay buffer or target networks. Instead, it computes advantage estimates via Generalized Advantage Estimation (GAE) and updates the policy using a clipped surrogate objective. This clipping mechanism restricts the magnitude of policy updates, preventing destructive steps and ensuring training stability. PPO serves as an ideal counterpoint to Double DQN, enabling a direct performance comparison between an off-policy value-based method and an on-policy policy-gradient method.

For PPO, the search space included the learning rate, GAE lambda, clipping parameter, entropy coefficient, value coefficient, rollout length, number of update epochs, mini-batch size and hidden-layer sizes.

4.3.3 Training Setup

After defining the Double DQN and PPO agents, a common training setup was used to ensure that the comparison between algorithms was fair and a random seed was set to ensure reproducibility.

Hyperparameter tuning was performed with Optuna over predefined search spaces. The candidate values were defined in advance, but the final configurations were selected automatically through the optimization process rather than by manual trial-and-error.

To reduce computational cost, the tuning stage used shorter runs than the final experiments. For each algorithm, Optuna was run for 20 trials. In each trial, a candidate configuration was trained for 10 000 episodes and evaluated over 500 episodes. During this process, configurations were compared using a multi-objective criterion that maximized both survival rate and mean

return.

Since the optimization involved two objectives, Optuna did not return a single best configuration directly. Instead, it produced a Pareto frontier containing the non-dominated configurations, representing the best trade-offs between survival rate and mean return. From this set, the final configuration was selected by prioritizing survival rate first and mean return second. This reflects the clinical priority of maximizing patient survival, while still accounting for the overall quality of the learned policy. After the tuning stage, the selected hyperparameter configuration for each algorithm was used to perform a full training run of 50 000 episodes.

5 Results and Evaluation

Figures and analysis.

5.1 Configuration A Results

5.1.1 Evaluation Results

Both Policy Iteration and Value Iteration produced identical results, achieving a mean return of 0.7865, a survival rate of 82.52%, and an average episode length of 9.85 steps. The similarity of the survival-rate curves shown in Figures 2 and 5 further confirms that both algorithms converged to the same treatment policy.

This outcome is expected, as both algorithms solve the same finite MDP and are guaranteed to converge to the optimal policy. The agreement between the two approaches provides additional confidence in the correctness of the implementation and indicates that the resulting policy represents the optimal solution for the environment (see Figures 2 and 5).

5.1.2 Convergence Analysis

Both algorithms converged successfully to the same optimal policy, but their convergence behavior differed considerably.

For Policy Iteration, the number of states changing action decreased rapidly from 386 in the first policy-improvement round to 85, then 5, before reaching zero in the fourth iteration (see Figure 3). This indicates that most policy improvements were identified early in the optimization process. Policy evaluation required between 172 and 194 iterations per round before convergence was achieved, resulting in a total of 736 value updates.

Value Iteration followed a more gradual convergence process. The maximum Bellman update error decreased from 0.7076 in the first iteration to below 0.0001 after 81 iterations (see Figure 6). The number of policy changes also declined steadily, reaching zero from iteration 61 onwards. The logarithmic representation in Figure 7 highlights the consistent reduction in value-function error across iterations and confirms stable convergence towards the optimal solution. While Value Iteration required substantially more iterations than Policy Iteration, both methods ultimately recovered the same policy.

5.1.3 Computational Performance

Despite producing identical policies, the algorithms differed considerably in runtime. As shown in Table 1, Policy Iteration completed in 5.78 seconds, whereas Value Iteration required 27.28 seconds. Consequently, Policy Iteration was approximately 4.7 times faster for this environment.

Given that both algorithms achieved identical evaluation results, Policy Iteration represents the more computationally efficient approach for Configuration A (see Table 1).

5.2 Configuration B Results

5.2.1 Training Curves

5.2.2 Performance Comparison

5.2.3 Deep RL Behavior Analysis

Learning stability, sample efficiency, generalization observations.

5.3 Configuration A vs Configuration B Comparison

Include comparison table, tabular RL vs deep RL, computational complexity, interpretability, stability, state representation differences, when deep RL becomes necessary.

6 Creative Extension: Policy Agreement & Ensemble Analysis

6.1 Motivation

As a creative extension, we explored policy agreement and ensemble decision-making in the continuous observation setting of Configuration B. While RL agents are commonly evaluated using aggregate performance metrics such as mean return or survival rate, these metrics provide limited insight into how different agents behave internally or whether they learn similar treatment strategies.

In clinical applications, robustness and consistency are particularly important. Two agents with comparable performance may still recommend substantially different treatments for the same patient state, raising questions regarding reliability and interpretability. Therefore, this extension aimed to investigate whether different RL algorithms converge toward similar clinical decisions and whether policy disagreement could be leveraged through ensemble methods to improve performance.

We hypothesized that agreement between agents would be higher in stable or low-severity patient states and lower in uncertain or clinically severe situations. Additionally, we expected that combining complementary behaviors from different policies through ensemble strategies could improve robustness and overall decision quality.

6.2 Method

After training both the Double DQN and PPO agents, evaluation trajectories were collected from each policy using identical episode seeds to ensure that both agents were evaluated under comparable clinical conditions. For every timestep, the stored information included the patient observation, SOFA severity score, selected action from each policy, action confidence scores, and indicators for noisy episodes or missing observations.

Policy agreement was defined as the percentage of timesteps in which both agents selected the same action. Agreement rates were analyzed globally and across different SOFA severity groups. To further study policy behavior throughout patient trajectories, episodes were divided into early and late phases, allowing agreement rates to be compared over time.

Disagreement analysis was then performed by identifying the most frequent disagreement action pairs between PPO and DDQN. Additional comparisons examined whether disagreement was associated with patient severity, noisy episodes, or missing observations.

Finally, two ensemble policies were implemented. Ensemble A followed a simple agreement-and-tie-break strategy: if both agents selected the same action, that action was executed; otherwise, the decision defaulted to a designated tie-break agent. Ensemble B implemented a confidence-based arbitration mechanism, selecting the action proposed by the agent with the highest confidence score. PPO confidence was obtained directly from policy probabilities, while DDQN confidence was approximated using a softmax transformation of the Q-values in order to place both confidence measures on a comparable scale.

6.3 Results and Discussion

The agreement analysis revealed extremely low policy agreement between DDQN and PPO. Across more than 17,000 evaluated timesteps, the agents selected the same action in only approximately 4% of states, indicating that they learned substantially different treatment strategies despite being trained in the same environment.

Contrary to the initial expectation, agreement remained consistently low across all SOFA severity groups (3.7%–4.5%), suggesting that disagreement was not strongly associated with patient severity (Figure 8). A small increase in agreement was observed during later episode stages, indicating that the agents may converge slightly more frequently as patient trajectories progress.

The disagreement analysis highlighted clear behavioral differences between the agents. PPO strongly concentrated its policy around a small subset of actions, particularly action 5, which accounted for more than 60% of selections. In contrast, DDQN distributed actions more broadly across the action space, producing a more diverse treatment policy. The action confusion matrix confirmed that most disagreements involved PPO repeatedly selecting action 5 while DDQN chose alternative interventions (Figure 8).

Agreement rates were only marginally affected by missing observations or noisy episodes, suggesting that disagreement was driven primarily by structural differences in learned policies rather than observation uncertainty.

The ensemble evaluation showed that combining policies could improve performance. Both

ensemble approaches outperformed the random baseline. Ensemble A achieved performance comparable to PPO, while Ensemble B obtained the highest mean return while maintaining a survival rate similar to the strongest individual policy (Figure 8). This suggests that confidence-based arbitration was more effective at leveraging complementary strengths between DDQN and PPO.

Overall, the extension demonstrates that RL agents can learn fundamentally different yet similarly effective treatment policies within the same clinical environment. The results also suggest that policy disagreement may contain complementary information that can be exploited through ensemble methods, highlighting the importance of robustness and interpretability in healthcare RL applications.

7 Discussion

- Key findings (best-performing methods, main differences between environments, exploration insights)
- Limitations (offline environment constraints, sparse reward, limited clinical realism, computational limitations, no real-world deployment validation)
- Future improvements (Reward shaping, safer RL, uncertainty-aware policies, better interpretability)

8 Conclusion

Restate project goal, summarize major findings, discuss importance of RL for healthcare, final reflection on discrete vs continuous RL.

References: Komorowski et al. (AI Clinician paper), Sutton and Barto (RL book), RL algorithm papers, MIMIC-III dataset paper, etc.

A Figures

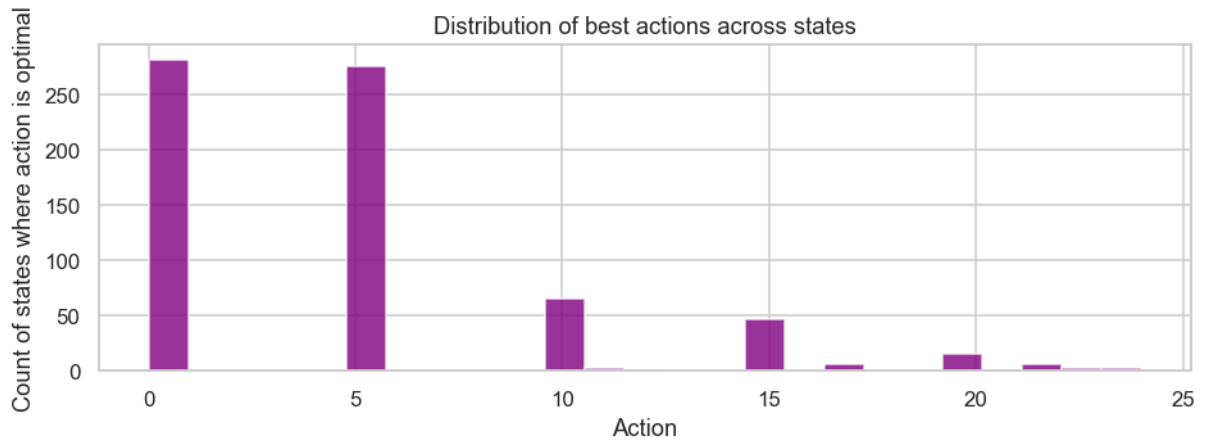


Figure 1: Distribution of Optimal Actions Selected Across All States.

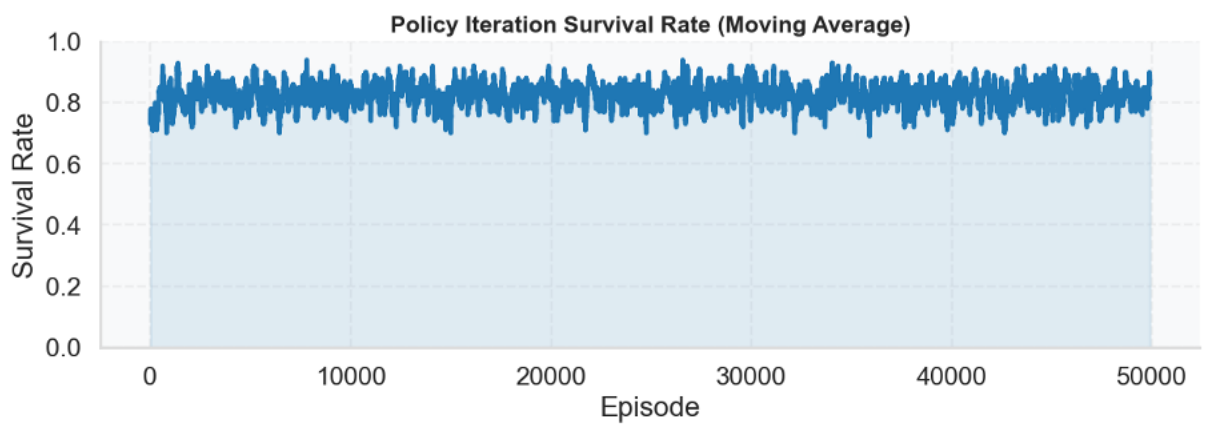


Figure 2: Policy Iteration Moving-Average Survival Rate over 50,000 Evaluation Episodes.

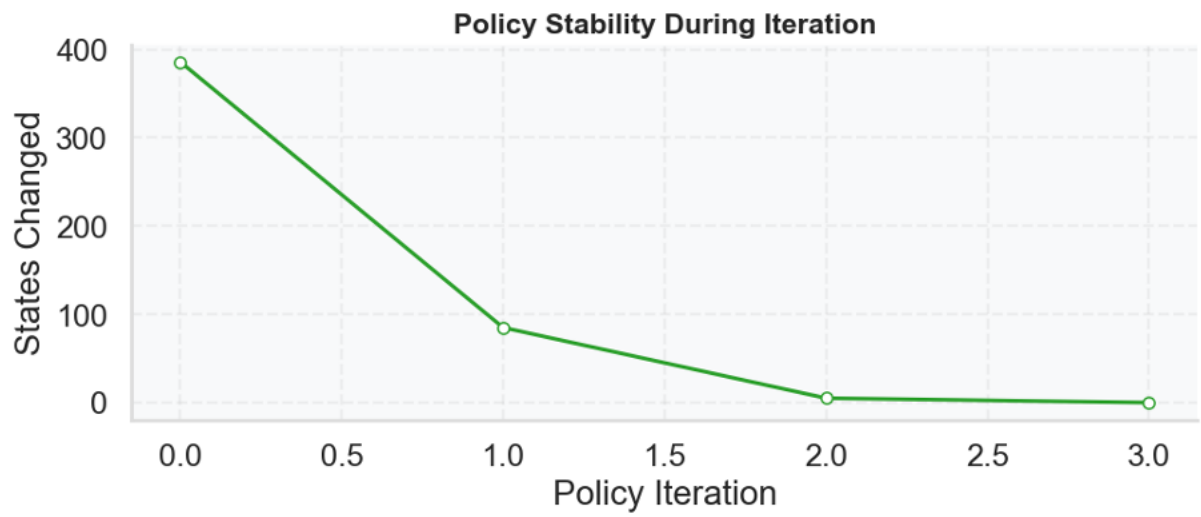


Figure 3: Number of States Changing During Successive Policy Iteration Improvement Steps.

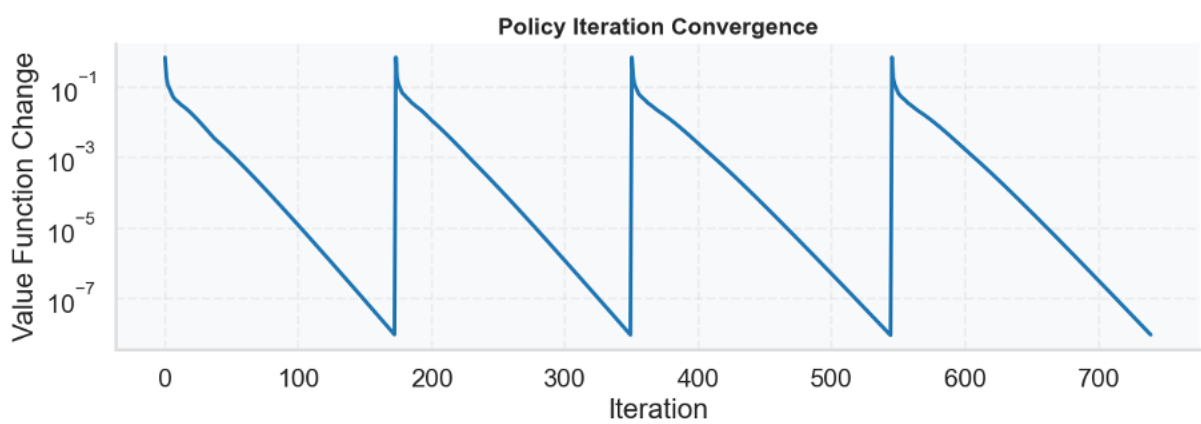


Figure 4: Convergence of the Value Function During Policy Iteration Shown on a Log Scale.

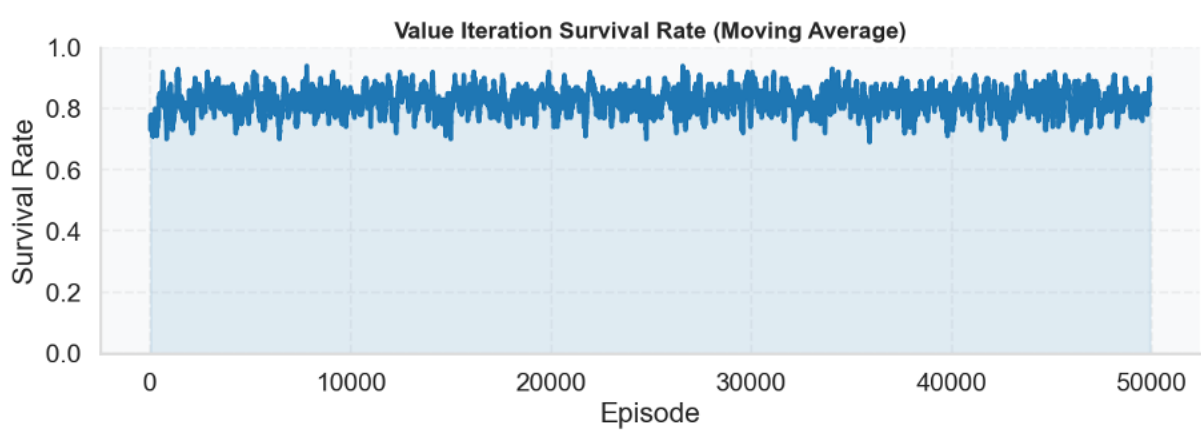


Figure 5: Value Iteration Moving-Average Survival Rate over 50,000 Evaluation Episodes.

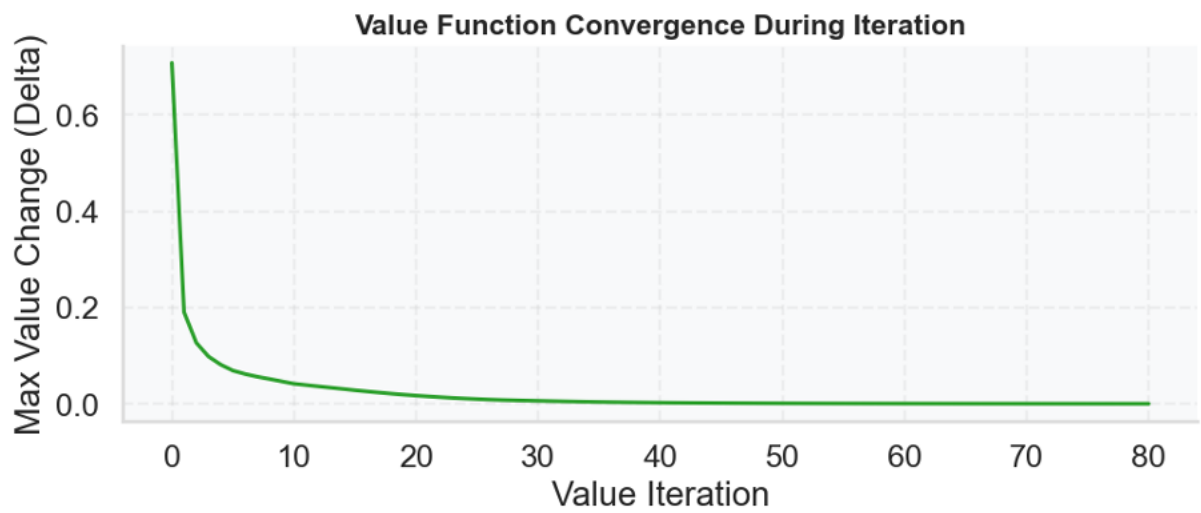


Figure 6: Decrease in the Maximum Bellman Update Difference During Value Iteration.

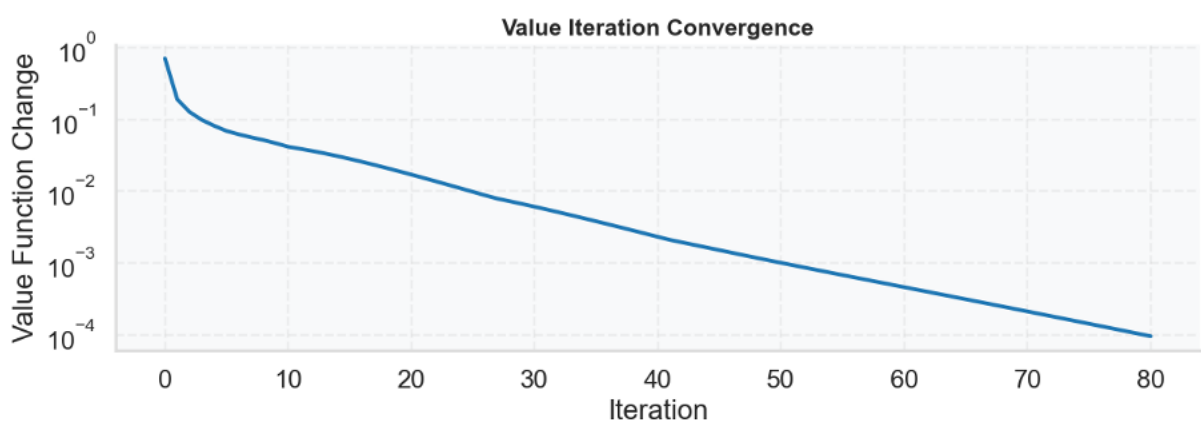


Figure 7: Log-Scale Convergence of the Value Function During Value Iteration.

Config B — Policy Agreement & Ensemble Analysis

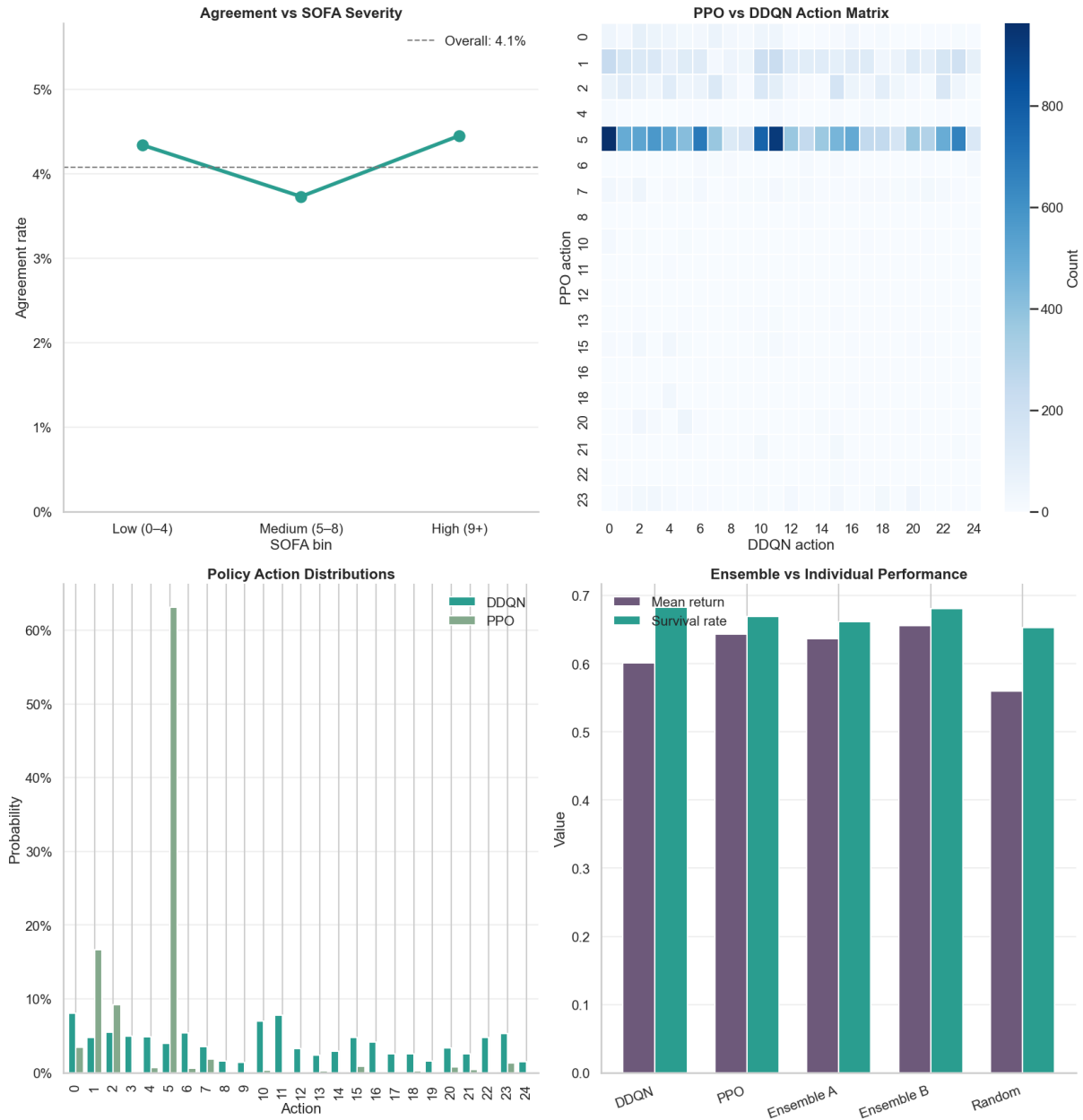


Figure 8: Comparison of Policy Agreement, Action Distributions, and Performance Metrics for DDQN, PPO, and Ensemble Policies under Configuration B.

B Tables

Table 1: Comparison of convergence performance between Policy Iteration and Value Iteration

Algorithm	Runtime (s)	Convergence Type	Convergence Count	Value Updates
Policy Iteration	5.7848	Policy improvement rounds	4	736
Value Iteration	27.2828	Bellman updates	81	81